

SUMMARY

Senior Systems Architect owning the backend platform and cloud infrastructure of a smart-home IoT product. Took over a legacy, manually deployed stack and turned it into a stable, high-ingestion platform: refactored legacy services, removed recurring outage causes at the root, and made zero-downtime migrations and controlled rollouts the default. Own the custom binary UDP transport the device fleet speaks today and drive R&D on its next-generation successor. Author the protocol specs and data contracts that firmware, mobile app, and backend teams build against, and propose architectural changes across those teams.

TECHNICAL COMPETENCIES

ARCHITECTURE	IoT Platform, Distributed Systems, High-Ingestion Pipelines, Binary Protocol Design, Zero-Downtime Migrations
LANGUAGES	Kotlin, Java, Go, Rust, Zig, C, Shell scripting (POSIX/Bash), SQL
KERNEL & NETWORKING	Linux kernel (<code>nftables</code> , <code>iptables</code>), in-kernel NAT64 (<code>Jool</code>), TCP/UDP socket programming, mTLS & custom PKI (CA/CRL), packet reflection, conntrack tuning
DISTRIBUTED & STORAGE	ClickHouse (columnar modeling, Delta+ZSTD codecs), Redis, PostgreSQL, MongoDB, gRPC, Protocol Buffers, REST
CLOUD & SRE	Google Cloud (IAM, VPC, Cloud Run, Cloud SQL, GCS), AWS (Lambda, EKS spike), Terraform / OpenTofu IaC, Docker, Proxmox VE, GitHub Actions, Linux administration (Debian, Ubuntu)
OBSERVABILITY	Grafana, Loki, VictoriaMetrics, VictoriaLogs, Alloy, <code>async-profiler</code> , JMC, VisualVM, Linux performance tools (<code>sar</code> , <code>vmstat</code> , <code>iostat</code> , <code>pidstat</code>)
METHODOLOGY	Extreme Programming (XP), London-Style TDD, Consumer-Driven Contract Testing (Pact spike), Testcontainers

PROFESSIONAL EXPERIENCE

eGlu Smart Homes (WiZNSystems)

Bengaluru, India · 2022 – Present

Senior Systems Architect (Apr 2026 – Present) | Senior Software Engineer (2024 – 2026) | Software Engineer (2022 – 2024)

IoT Platform & High-Ingestion Protocols

- Custom UDP Transport for the Device Fleet:** Define and maintain the custom binary UDP protocol that smart-home hubs use to talk to the cloud, and the high-ingestion backend that decodes it. Tuned the ingest path end to end for throughput and low per-packet latency, from kernel-level packet handling to a load-adaptive JVM thread pool and allocation-free packet decoding.
- Next-Generation Protocol R&D:** Driving R&D on the next-generation device transport protocol to succeed the current UDP transport.
- Cross-Team Protocol & Architecture Proposals:** Author the binary protocol definitions, API contracts, and technical specs (Git-based knowledge base) that firmware, mobile app, and backend teams review and implement against before releases. Propose architectural changes to the firmware and app teams, including the SNode virtual-node abstraction (aggregating disjoint physical devices into one composite device) and backward-compatible mobile rollout contracts.

Legacy Revamp & Platform Stabilization

- Legacy Platform Modernization:** Took full technical ownership of the backend and cloud infrastructure in 2023. Replaced manual SFTP WAR deployments with automated CI/CD, migrated Spring 4 (JDK 8) to Spring Boot 2.7.5 (JDK 17) to unlock TDD, rewrote the device provisioning and WiFi onboarding state machine, and removed obsolete dynamic TCP port provisioning and redundant in-memory state tracking.
- Stabilizing Production:** Eliminated recurring outages at the root: systemd file-descriptor limits starving thread allocation and locking out SSH, an auxiliary service leaking unpooled MongoDB connections, and silent "ghost connections" poisoning Redis Pub/Sub pools (Lettuce vs. Jedis). Fixed a months-old legacy sync bug silently corrupting hub automations and reconciled state across thousands of live homes without breaking customer automations.
- Reducing Downtime with Deliberate Release Strategy:** Made zero-downtime the default: kernel-level blue-green cutovers (`iptables` DNAT across PREROUTING and OUTPUT , `conntrack` flush), atomic, idempotent deploy tooling with pre-flight health checks and rollback guards, a mandatory on-prem staging gate mirroring production, and a feature flags engine (percentage rollouts, kill switches) for risky cloud, mobile, and firmware changes.

Systems Architecture & Kernel Networking

- In-Kernel Dual-Stack NAT64 (`Jool` + `nftables`):** Resolved an edge network partition where cellular IoT hubs operated strictly over IPv6 while backend services were IPv4. Bypassed userspace proxies to preserve CPU credits; deployed the `Jool` kernel module for stateful NAT64 alongside `nftables` DNAT to cleanly partition source port ranges on a shared public IPv4.
- Prerouting Packet Reflection & Dynamic Abuse Mitigation:** Offloaded UDP NAT traversal echo responses directly into the Linux kernel using `nftables` packet reflection (`notrack`) at prerouting priority, avoiding userspace context switches. Implemented kernel-level DDos mitigation via `nftables` dynamic sets (`limit rate over 10/second burst 20 packets` at prerouting priority -301) dropping sweeps before socket allocation.
- Academic Preprint Publication (Zenodo):** Authored and published *Connection-Agnostic Presence Tracking for Stateless Distributed Backends*: formulated a Redis sorted-set architecture with expiration deadlines and throttled batch writes to track IoT device state with bounded detection latency.

High-Throughput Telemetry & Storage Systems

- **ClickHouse Ingestion Engine (95%+ Compression):** Resolved filesystem inode exhaustion caused by legacy directory log dumps. Designed high-throughput columnar ClickHouse schemas (`FixedString` , `LowCardinality` , Delta + ZSTD codecs, sparse composite primary keys), slashing telemetry footprint by 95%+ (compressing `device_health` telemetry from 17 GB down to 300 MB on disk). Executed zero-downtime partition-level backfilling and restore operations without slow `INSERT INTO` batches, and devised partition-level disaster recovery backup and restore policies.
- **Decoupled Ingestion Microservice:** Engineered an auxiliary Go microservice communicating via binary gRPC with strict Protocol Buffers contracts to encapsulate ClickHouse, shielding the primary Kotlin/Spring Boot monolith from database driver coupling.
- **Zero-Downtime Storage Architecture:** Architected and orchestrated an end-to-end multi-phase data migration decoupling binary Base64 image blobs from primary MongoDB collections to GCS with zero downtime; designed dual-write API contracts with document guardrails (`imageMigrated`), executed an asynchronous backfill pipeline with immutable GCS caching headers, coordinated backward-compatible mobile rollouts, and purged legacy blobs via `$unset` , slashing database storage by 54x (3.6 GB to 67 MB) and runtime memory by 89% (4.6 GB to 500 MB).
- **Production MongoDB DBA & Index Engineering:** Acted as hands-on DBA across heavy-load production MongoDB clusters. Audited slow query logs and execution plans (`explain`), pruned redundant/overlapping indexes to reclaim WiredTiger cache memory and reduce write amplification, refactored hot document schemas, and engineered selective compound indexes (strict Equality-Sort-Range ordering) to eliminate collection scans and in-memory sorts under sustained IoT write traffic.

Concurrency, Runtime & JVM Performance

- **Dynamic IoT-Aware ThreadPool:** Overhauled JVM thread management under bursty IoT packet waves by replacing default thread pools with an adaptive dynamic ThreadPool that scales worker threads based on real-time packet velocity and queue depth.
- **Deep JVM Profiling & Zero-Allocation Hot Paths:** Instrumented live production systems with `async-profiler` and JMC flame graphs. Mitigated allocation churn with buffer pooling and `ThreadLocal` storage; refactored critical packet decoding loops from `String.format` to Java 17 `HexFormat` and bitwise arithmetic, eliminating hot-path CPU bottlenecks. Pinned heap boundaries (`-Xms = -Xmx` at 4GB) and tuned GC for low pause times.

Platform Engineering, Governance & Cloud FinOps

- **Google Cloud Platform Administration & FinOps:** Google Cloud Platform Administrator managing IAM policies, VPC networks, Cloud Storage, and compute infrastructure. Evaluated spend-based vs. resource-based CUD models and executed a 3-year resource-based Committed Use Discount (CUD) on core compute instances.
- **Observability & Telemetry Infrastructure:** Replaced ad-hoc raw log dumping on production VM disks with centralized log aggregation using Grafana and Loki. Built a pre-production Docker Compose telemetry stack integrating Grafana, Loki, VictoriaMetrics, VictoriaLogs, and Alloy with host and container performance dashboards.
- **Zero-Trust Diagnostic Tooling (`hlogger`):** Built a secure client-server log inspection utility in Go using Mutual TLS (mTLS) with a custom PKI hierarchy (Root CA, Intermediate CA, client certificates, CRL), eliminating unauthenticated engineer SSH access to production instances.
- **Bare-Metal Homelab:** Configured a Proxmox VE bare-metal host provisioned via OpenTofu and `cloud-init` to evaluate container orchestration (k3s, HashiCorp Nomad, AWS EKS spikes).
- **Declarative Marketing Infrastructure:** Decoupled the public marketing website completely from core backend infrastructure into its own isolated GCP project, codifying 100% of Cloud Run, Cloud SQL (PostgreSQL), networking, and IAM via Terraform / OpenTofu.
- **Platform Automation & Integrations:** Extended scene/rule automation engines for multi-hub environments (fragment handling, cross-hub synchronization); built cloud-to-cloud OAuth integrations for Yale smart locks, Google Home, and Alexa; built replacement operations for failed hardware nodes without losing room mappings or automations.
- **Technical Hiring:** Formulated structured, multi-step technical hiring assessments and evaluation rubrics for backend engineering; interviewed candidates and hired software engineering talent for the platform.

OPEN-SOURCE SYSTEMS & RESEARCH

stopgap Kotlin · Maven Central (`2.8.0`)

Microservice framework on Helidon SE (Nima) + Project Loom virtual threads. Custom compile-time DI via KSP (zero runtime reflection). Three-tier testing harness: unit → in-process integration server → Docker E2E via Testcontainers.

assertgo Go · Generics

Type-safe testing assertion library built with modern Go generics. Provides a fluent assertion API, chainable `Not()` , custom matchers, and zero external dependencies.

relay Zig · Systems

Low-level TCP server in pure Zig: raw POSIX sockets, manual memory management, no libc runtime, `SIGPIPE` suppression.

hrh Rust · CLI

Helm Release Helper CLI — automated declarative Helm deployments with diff previews and atomic rollback guarantees via `cargo install` .

avoid Shell / Linux · Distro

Minimal, bootable Linux distribution based on Void Linux engineered for server disaster recovery and lightweight headless appliances.

EDUCATION

Bachelor of Computer Applications (BCA)

2019 – 2022